

Parallel Stochastic Filtering

From Shared-Memory Particle Filtering to Hybrid Parallel Methods

Siddhartha Gupte
Parallel Computing

September 21, 2026

Abstract

This project studies stochastic filtering as a progressively more challenging parallel-computing problem. Beginning with a linear-Gaussian state-space model and an exact Kalman-filter benchmark, a bootstrap particle filter is developed, validated, and parallelized using OpenMP. Subsequent stages extend the framework to nonlinear filtering, parallel resampling, coupled fine-coarse simulations, distributed-memory execution with MPI, and hybrid MPI/OpenMP methods. The emphasis is on both numerical accuracy and computational scalability, with particular attention to synchronization, reductions, communication, and serial bottlenecks.

Contents

1	Introduction	2
2	Background	2
2.1	State-Space Models and Bayesian Filtering	2
2.2	Kalman Filtering	3
2.3	Particle Filtering	3
2.4	Parallel Structure	4
3	Stage I: Linear-Gaussian Benchmark and Shared-Memory Particle Filtering	5
3.1	Stage-I Setup	5
3.2	Numerical Validation	6
3.3	Parallelization Strategy	8
3.4	Synchronization and Reductions	9
3.5	Strong-Scaling Results	10
3.6	Where the Scaling Stops	11
3.7	Stage-I Summary	13

1 Introduction

Stochastic filtering concerns the estimation of a time-varying hidden state from noisy observations. The problem appears in many settings where the quantity of interest cannot be measured directly but must instead be inferred sequentially as new data arrive.

For linear systems with Gaussian noise, the filtering distribution can be computed exactly using the Kalman filter. More general models typically require numerical approximations. Particle filtering provides one such approach by representing the filtering distribution with a collection of weighted samples, or particles. Increasing the number of particles improves the Monte Carlo approximation, but also increases the computational cost.

Particle filters are particularly interesting from a parallel-computing perspective. Propagation and likelihood evaluation can be carried out independently across particles, while operations such as normalization, estimation, and resampling require some form of global coordination. The algorithm therefore contains both naturally parallel work and potential scalability bottlenecks.

This project studies that interaction between filtering algorithms and parallel computation. The work is organized in stages, with each stage building on the previous implementation.

Stage I uses a scalar linear-Gaussian model for which the Kalman filter provides an exact benchmark. A bootstrap particle filter is first validated against this reference solution and is then parallelized using OpenMP. The resulting implementation is used to study race conditions, reductions, strong scaling, and the effect of serial components on end-to-end performance.

Later stages will extend the filtering and parallel-computing framework as the project develops. The common questions are how accurately the filter approximates the desired state distribution, which parts of the algorithm can be parallelized effectively, and which operations ultimately limit scalability.

Section 2 summarizes the filtering concepts needed for Stage I. The remaining sections describe the individual stages, their implementations, and their numerical and performance results.

2 Background

2.1 State-Space Models and Bayesian Filtering

A discrete-time state-space model consists of an unobserved state X_t and an observation Y_t ,

$$X_t \sim p(x_t \mid x_{t-1}), \tag{1}$$

$$Y_t \sim p(y_t \mid x_t). \tag{2}$$

The filtering problem is to compute the conditional distribution

$$p(x_t \mid y_{0:t}), \tag{3}$$

where $y_{0:t}$ denotes all observations available through time t .

The recursion consists of a prediction step,

$$p(x_t \mid y_{0:t-1}) = \int p(x_t \mid x_{t-1}) p(x_{t-1} \mid y_{0:t-1}) dx_{t-1}, \tag{4}$$

followed by a correction after observing y_t ,

$$p(x_t \mid y_{0:t}) \propto p(y_t \mid x_t) p(x_t \mid y_{0:t-1}). \tag{5}$$

2.2 Kalman Filtering

For a linear-Gaussian model,

$$X_t = AX_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, Q), \quad (6)$$

$$Y_t = CX_t + \eta_t, \quad \eta_t \sim \mathcal{N}(0, R), \quad (7)$$

the filtering distribution remains Gaussian. It is therefore sufficient to update its mean and variance.

If m_t^- and P_t^- denote the predictive mean and variance, the scalar Kalman update is

$$K_t = \frac{P_t^- C}{C^2 P_t^- + R}, \quad (8)$$

$$m_t = m_t^- + K_t(Y_t - C m_t^-), \quad (9)$$

$$P_t = (1 - K_t C) P_t^-, \quad (10)$$

with prediction

$$m_{t+1}^- = A m_t, \quad (11)$$

$$P_{t+1}^- = A^2 P_t + Q. \quad (12)$$

For the linear-Gaussian model used in Stage I, these recursions provide the exact filtering mean and therefore a convenient benchmark.

2.3 Particle Filtering

A particle filter approximates the filtering distribution using weighted samples,

$$p(x_t \mid y_{0:t}) \approx \sum_{i=1}^N w_t^{(i)} \delta_{X_t^{(i)}}(x_t), \quad (13)$$

with normalized weights satisfying

$$\sum_{i=1}^N w_t^{(i)} = 1. \quad (14)$$

In a bootstrap particle filter, particles are propagated using the state transition,

$$X_t^{(i)} \sim p(x_t \mid X_{t-1}^{(i)}), \quad (15)$$

and weighted according to the observation likelihood,

$$\tilde{w}_t^{(i)} = p(y_t \mid X_t^{(i)}). \quad (16)$$

After normalization,

$$w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{j=1}^N \tilde{w}_t^{(j)}}, \quad (17)$$

quantities such as the filtered mean can be estimated by

$$\hat{X}_t = \sum_{i=1}^N w_t^{(i)} X_t^{(i)}. \quad (18)$$

The particle population is then resampled according to the normalized weights. Since the method is Monte Carlo, its statistical error is expected to decrease at the usual $O(N^{-1/2})$ rate under standard conditions.

2.4 Parallel Structure

Particle propagation and likelihood evaluation are independent across particles and are therefore natural targets for data parallelism. Normalization and posterior estimation require reductions over the particle population, while resampling depends on the full set of weights.

The distinction between particle-local work and global operations is the main computational issue considered in Stage I.

3 Stage I: Linear-Gaussian Benchmark and Shared-Memory Particle Filtering

The first stage serves two purposes. First, it gives us a filtering problem for which the correct answer is known: the linear-Gaussian model can be solved exactly with a Kalman filter. Second, it gives us a simple particle filter in which the sources of parallelism are easy to isolate.

The main question in this stage is therefore not only whether the particle filter is numerically correct, but also which parts of the algorithm benefit from shared-memory parallelism and which parts prevent the full filter from scaling.

3.1 Stage-I Setup

We use the scalar state-space model

$$X_t = 0.9X_{t-1} + 0.5\varepsilon_t, \quad (19)$$

$$Y_t = X_t + \eta_t, \quad (20)$$

with

$$\varepsilon_t, \eta_t \sim \mathcal{N}(0, 1), \quad X_0 \sim \mathcal{N}(0, 1). \quad (21)$$

The simulation parameters are collected in [table 1](#).

Table 1: Parameters used in the Stage-I state-space simulation.

Parameter	Value
Number of time steps T	500
State coefficient A	0.9
Observation coefficient C	1.0
State-noise standard deviation σ_x	0.5
Observation-noise standard deviation σ_y	1.0
Initial mean μ_0	0.0
Initial standard deviation σ_0	1.0

A single trajectory of length $T = 500$ was generated and reused in all experiments. The latent state is known because the data are simulated, but neither filter receives it as input; both filters see only the observations Y_t .

For this model,

$$Q = \sigma_x^2 = 0.25, \quad R = \sigma_y^2 = 1, \quad (22)$$

so the Kalman filter described in [section 2.2](#) gives the exact filtering mean.

The bootstrap particle filter follows the usual cycle

$$\text{propagate} \rightarrow \text{weight} \rightarrow \text{normalize} \rightarrow \text{estimate} \rightarrow \text{resample}. \quad (23)$$

For the Gaussian observation model, the unnormalized particle weight is

$$\tilde{w}_t^{(i)} = \exp \left[-\frac{(Y_t - CX_t^{(i)})^2}{2\sigma_y^2} \right], \quad (24)$$

and the filtered estimate is

$$\hat{X}_t^{\text{PF}} = \sum_{i=1}^N w_t^{(i)} X_t^{(i)}. \quad (25)$$

Multinomial resampling is used after each update. The state-space simulation used seed 42, while the particle filter used a separate seed 12345.

3.2 Numerical Validation

Before parallelizing the particle filter, we first checked that it was solving the filtering problem correctly.

Figure 1 shows the simulated state together with the noisy observations and the Kalman estimate.

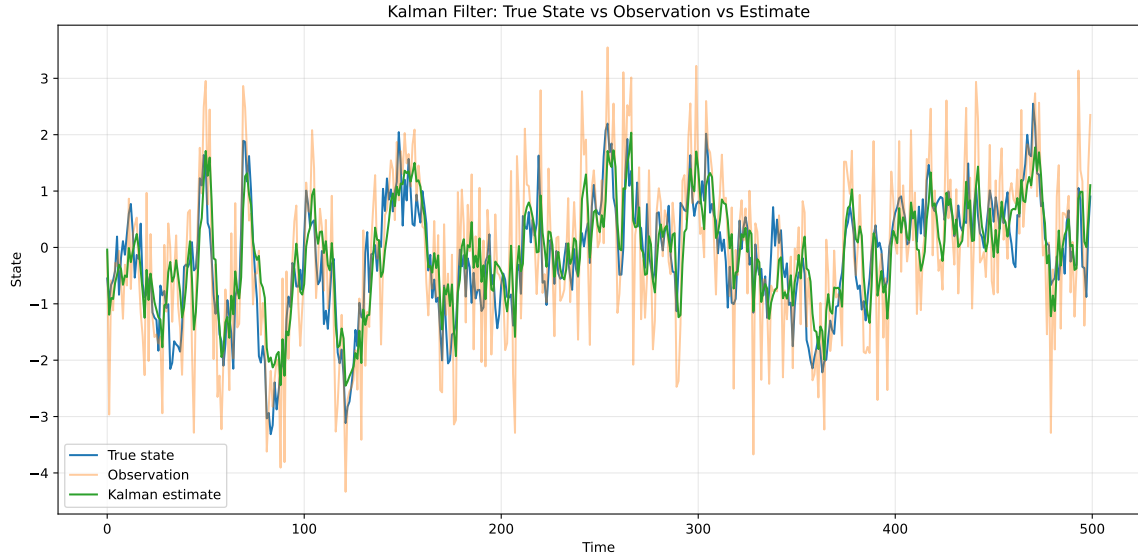


Figure 1: Simulated state, observations, and Kalman-filter estimate.

The raw observations have an RMSE of 1.029944 relative to the true state, while the Kalman estimate has an RMSE of 0.616717.

A particle filter with $N = 10,000$ particles gives nearly the same trajectory as the Kalman filter, as shown in figure 2.

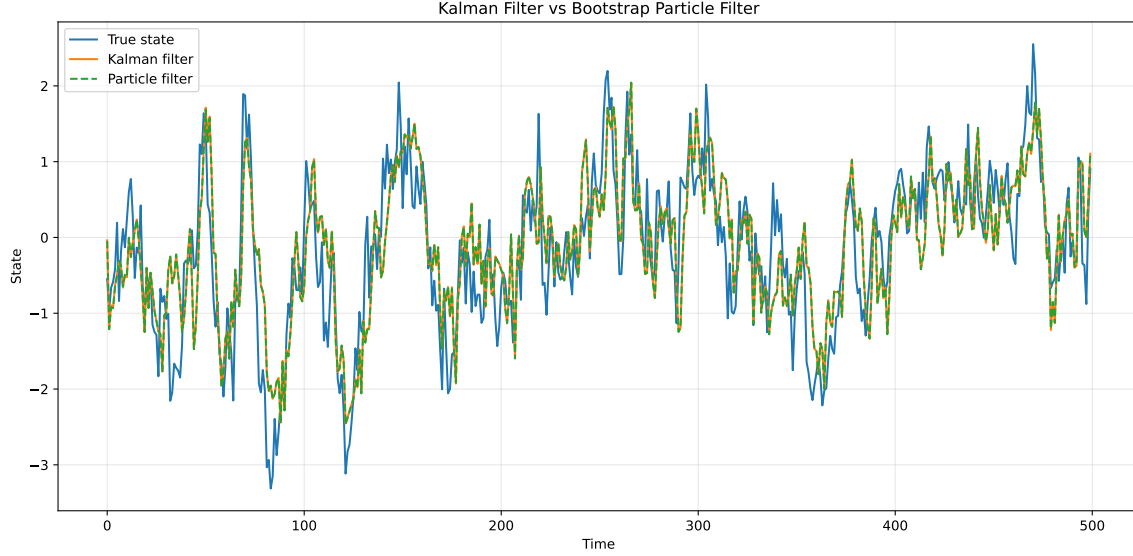


Figure 2: Kalman and bootstrap particle-filter estimates for $N = 10,000$.

The main numerical checks are summarized in [table 2](#).

Table 2: Numerical validation of the serial particle filter.

Quantity	RMSE
Observations vs. true state	1.029944
Kalman vs. true state	0.616717
PF vs. true state, $N = 10,000$	0.617360
PF vs. Kalman, $N = 10,000$	0.009906

The more direct test of the particle approximation is the PF–Kalman difference, since the Kalman mean is exact for this model. We therefore repeated the serial particle filter over particle counts ranging from 100 to 500,000.

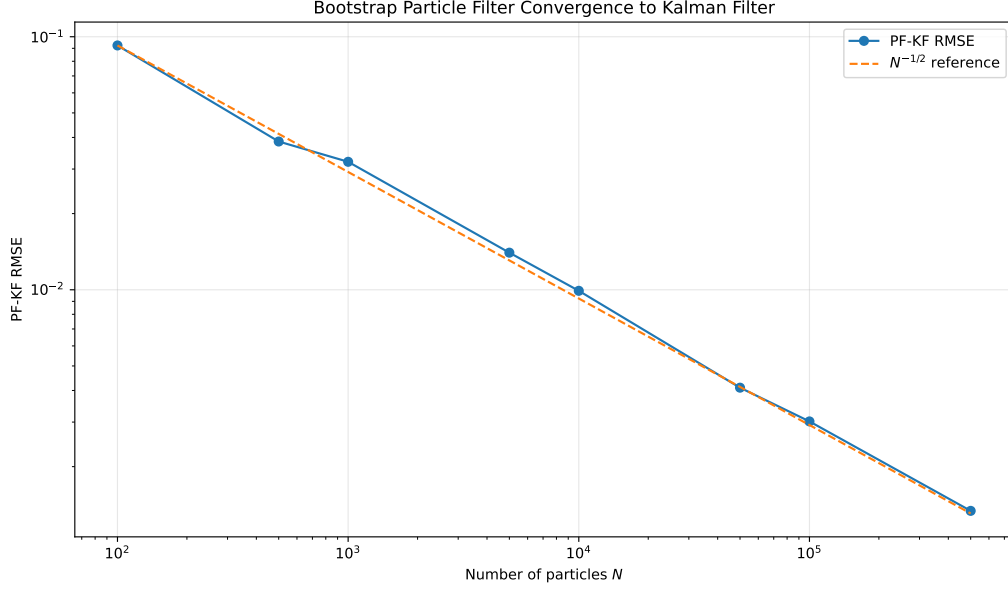


Figure 3: PF–Kalman RMSE versus particle count. The dashed curve shows an $N^{-1/2}$ reference rate.

The log–log slope of the measured PF–Kalman RMSE is approximately

$$-0.497, \quad (26)$$

which is close to the expected Monte Carlo rate of $-1/2$. This gives us confidence that the serial implementation is behaving as expected before introducing concurrency.

3.3 Parallelization Strategy

The particle index provides the natural source of data parallelism. Once the particle population at time $t - 1$ is available, particle i can be propagated independently,

$$X_t^{(i)} = AX_{t-1}^{(i)} + \sigma_x \varepsilon_t^{(i)}, \quad (27)$$

and its likelihood can also be evaluated without reference to any other particle. These loops were therefore parallelized with OpenMP using static scheduling.

Not every step is particle-local, however. Weight normalization requires

$$Z_t = \sum_{i=1}^N \tilde{w}_t^{(i)}, \quad (28)$$

and the posterior mean requires

$$\hat{X}_t^{\text{PF}} = \sum_{i=1}^N w_t^{(i)} X_t^{(i)}. \quad (29)$$

Both are global reductions.

Random-number generation also needs some care. Sharing a single `std::mt19937` engine between threads would itself require synchronization. Instead, the implementation creates one generator per OpenMP thread. Each thread accesses only its own engine while propagating its assigned particles.

The Stage-I parallel structure is summarized in [table 3](#).

Table 3: Parallel structure of the Stage-I particle filter.

Operation	Implementation
Particle propagation	Parallel
Likelihood evaluation	Parallel
Weight summation	OpenMP reduction
Weight normalization	Parallel
Posterior mean	OpenMP reduction
Multinomial resampling	Serial

The last row is important. Resampling was deliberately left serial in Stage I. This lets us study the first layer of OpenMP parallelism without also changing the resampling algorithm.

3.4 Synchronization and Reductions

The weight sum provides a useful small experiment in parallel correctness. Consider the apparently harmless loop

```
#pragma omp parallel for
for (...) {
    weight_sum += weights[i];
}
```

The addition is a read–modify–write operation on shared memory. Two threads can read the same old value of `weight_sum`, compute different updates, and then overwrite one another. The result is a race condition.

We tested three implementations of the same sum using $N = 100,000$ particles and 16 threads. For one fixed set of weights, the correct sum was

$$70562.5. \tag{30}$$

The naive implementation returned different values on repeated runs, including 8786.29, 13206.7, and 13243.8.

A `critical` section fixes the race by allowing only one thread at a time to update the accumulator. It is correct, but it also turns the accumulation into a serial bottleneck. An OpenMP reduction avoids this by giving each thread a private partial sum and combining those partial sums at the end.

Table 4: Weight-summation experiment for $N = 100,000$ and $p = 16$.

Method	Correct?	Result	Observed cost
Naive shared sum	No	Nondeterministic	–
<code>critical</code>	Yes	70562.5	8–12 ms
OpenMP reduction	Yes	70562.5	20–70 μ s

This experiment is a useful example of the distinction between *thread safety* and *parallel efficiency*. The critical section makes the program correct, but correctness is achieved by serializing every update. The reduction gives both correctness and a parallel accumulation strategy.

3.5 Strong-Scaling Results

The strong-scaling tests were run on a 13th Gen Intel Core i7-13620H with 10 physical cores and 16 logical CPUs. The code was compiled with GCC 16.1.1 using C++23, OpenMP, and a Release build.

We tested

$$N \in \{10^3, 10^4, 10^5, 5 \times 10^5\} \quad (31)$$

with

$$p \in \{1, 2, 4, 8, 16\} \quad (32)$$

OpenMP threads. Each configuration was run five times, and the median wall-clock runtime was used.

Speedup and efficiency are defined by

$$S_p = \frac{T_1}{T_p}, \quad E_p = \frac{S_p}{p}. \quad (33)$$

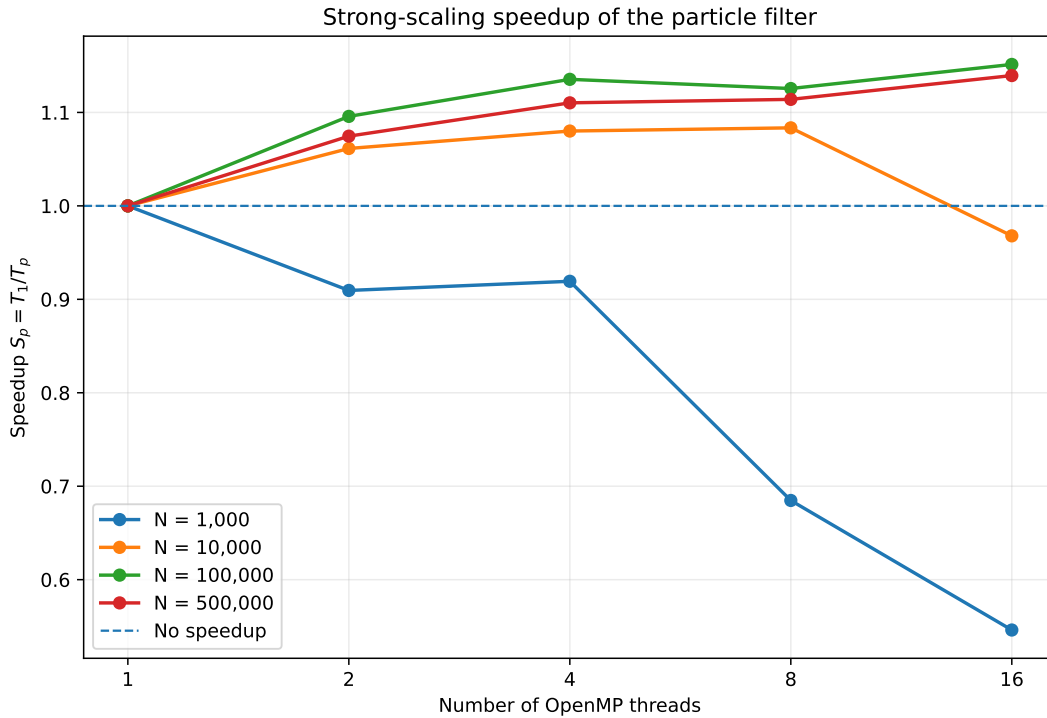


Figure 4: Strong-scaling speedup for different particle counts. The dashed line marks the serial baseline $S_p = 1$.

The behavior depends strongly on problem size. At $N = 1,000$, the OpenMP overhead is larger than the work saved by parallel execution, so adding threads actually slows the filter down. By $N = 10,000$, a small speedup appears. For the two largest populations, the improvement is more consistent, although still modest.

A few representative values are shown in [table 5](#).

Table 5: Strong-scaling results at 16 OpenMP threads.

N	Threads	Speedup	Efficiency
1,000	16	0.546	0.034
10,000	16	0.968	0.060
100,000	16	1.151	0.072
500,000	16	1.139	0.071

The best end-to-end speedup is only about 1.15, despite using 16 threads.

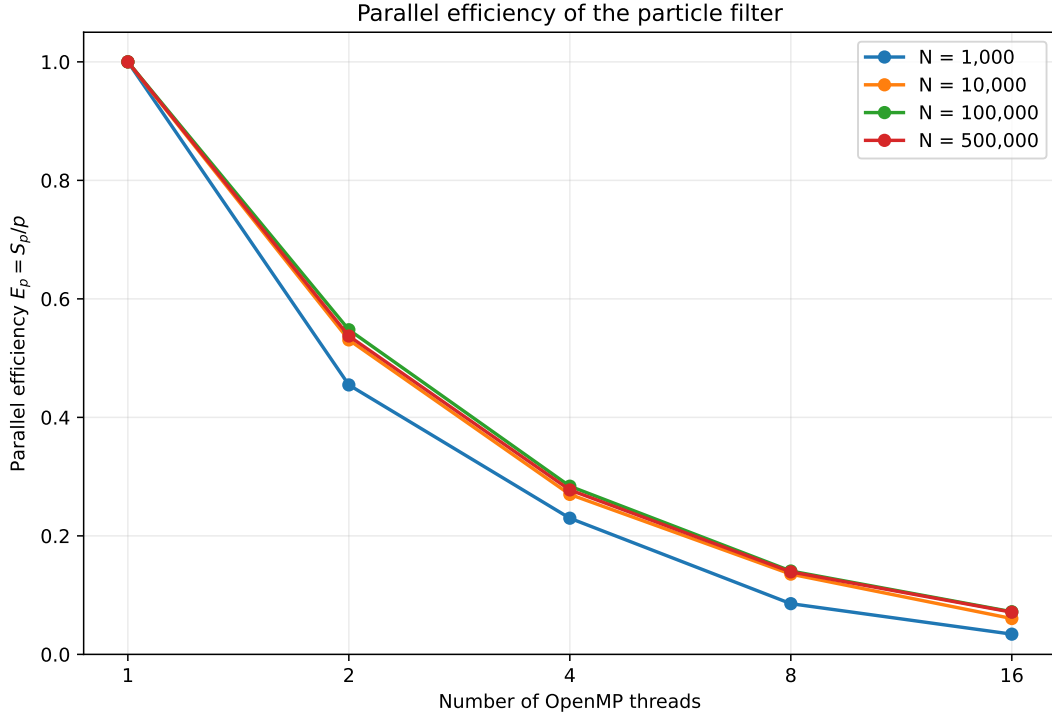


Figure 5: Parallel efficiency versus OpenMP thread count.

The efficiency plot makes the same point from another angle. Even for large particle populations, efficiency drops rapidly as threads are added. This suggests that the limitation is not simply insufficient particle-level work; some part of the full filter is not scaling with the OpenMP kernels.

3.6 Where the Scaling Stops

To find the bottleneck, we timed the major pieces of the $N = 500,000$ filter separately.

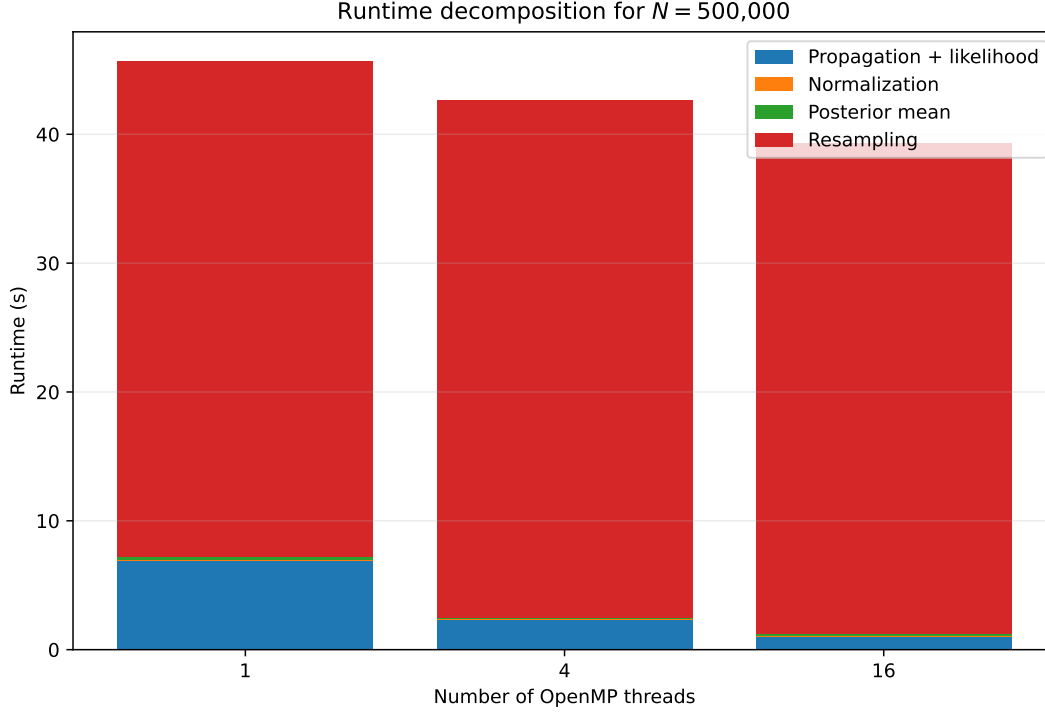


Figure 6: Runtime decomposition for $N = 500,000$.

The raw measurements are given in [table 6](#).

Table 6: Runtime decomposition for $N = 500,000$, in seconds.

Threads	Prop.+likelihood	Normalization	Posterior mean	Resampling	Total
1	6.876	0.142	0.158	38.494	45.945
4	2.327	0.094	0.089	40.156	42.961
16	1.014	0.120	0.080	38.057	39.566

The interesting result is that the OpenMP kernel itself scales reasonably well. Propagation and likelihood evaluation fall from 6.88 s to 1.01 s between one and sixteen threads, a speedup of about

$$\frac{6.87557}{1.01447} \approx 6.78. \quad (34)$$

The full filter does not inherit that speedup because multinomial resampling remains serial. Its runtime stays close to 38–40 s regardless of the number of threads.

At one thread, resampling accounts for about 84% of the total runtime. At sixteen threads, after the particle-local work has been accelerated, that fraction rises to about 96%.

This is precisely the situation described by Amdahl’s law. If f is the serial fraction,

$$S_p \leq \frac{1}{f + (1 - f)/p}. \quad (35)$$

Taking $f \approx 0.84$ from the one-thread profile gives the rough asymptotic limit

$$S_{\infty} \approx \frac{1}{0.84} \approx 1.19. \quad (36)$$

The observed end-to-end speedup of about 1.14–1.15 is therefore not surprising. The parallel kernel is getting faster; it is simply becoming a smaller and smaller fraction of the total runtime.

3.7 Stage-I Summary

Stage I established a numerical and computational baseline for the rest of the project.

The bootstrap particle filter converged toward the exact Kalman solution at approximately the expected $N^{-1/2}$ Monte Carlo rate.

The OpenMP implementation also exposed several basic shared-memory issues directly. Particle-local work is straightforward to distribute, random-number generators must be thread-local, and global sums require proper reductions rather than unsynchronized shared updates. The comparison between the naive sum, `critical`, and `reduction` gave a concrete example of the difference between correct synchronization and scalable synchronization.

Finally, the performance results showed why parallelizing only the obvious loops is not enough. The propagation and likelihood kernel achieved about $6.8\times$ speedup at sixteen threads, while the complete filter achieved only about $1.15\times$. Serial resampling was responsible for most of that gap.

The main outcome of Stage I is therefore not simply an OpenMP particle filter, but a clear picture of where its parallelism comes from and where the current implementation stops scaling.